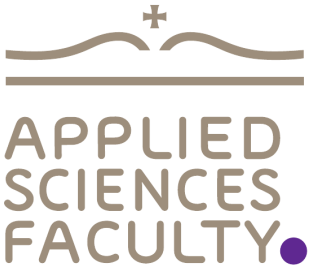
C++ — лекція 6. Функції — 1, основи.





©Олег Фаренюк,
Creative Commons - Attribution Share Alike (CC BY-SA)

Функції

- Весь код програми на C/C++ знаходиться в функціях.
- Функція — іменований блок коду, який можна викликати, і який після виконання своїх задач повертає керування у місце виклику.
- Іншими словами, функція — вид підпрограми. C++ не розділяє поняття функція і процедура, як це роблять деякі інші мови.
- Тут говоримо про "звичайні" функції, крім них в C++ є лямбда-функції, об'єкти функції та результати роботи bind().
- ◇ Всі ці сутності можна викликати як функції, тому їх збірно називають **callable**. Вивчатимемо їх трохи пізніше – вони є узагальненнями ідеї функції.

Оголошення функції

Перш ніж використовувати функцію, її слід оголосити (declaration) — задати її прототип.

- Оголошення вказує компілятору, як слід викликати функцію: що вона повертатиме, які аргументи може приймати.
- Завдяки оголошенню компілятор може перевірити передані аргументи, і, якщо потрібно, виконати неявні приведення типів, щоб задовільнити вимоги **прототипу** функції.
- Приклади оголошення:

```
int multipy_by_2(int x);
void exit(int code);
char* strcpy(char* to, const char* from);
```

- Оголошень функції може бути скільки завгодно.

Визначення функції

- Щоб програма могла реально викликати функцію, вона повинна мати визначення (definition) — код, який виконуватиметься під час виклику цієї функції.
- Визначення функції повинно бути одне і тільки одне на програму[1]!
- Оголошення функції – це її визначення, після якого, **замість** крапки з комою, йде тіло функції, у фігурних дужках.

```
int multipy_by_2(int x){
    int res;
    res = 2 * x;
    return res;
}
```

- Типи параметрів та значення, що повертається, повинні співпадати у всіх оголошеннях та у визначенні.

[1] Виняток – методи, визначені в оголошенні класів, шаблони, inline-функції і т.д., але і там працює ODR – One Definition Rule, якщо є кілька визначень – вони повинні співпадати. Детальніше див. https://en.cppreference.com/w/cpp/language/definition.

Відсутність вкладених функцій

- Визначення функцій повинно бути за межами інших функцій — вкладені функції заборонені[2].
- ◇ Однак, оголошення можуть знаходитися в інших функціях:

```
void some_function(){
    void fn(int, int); //! Функція є в іншій одиниці трансляції
    fn(3, 14);
}
```

- Однак, у C++ можливо майже все – схожий ефект можна отримати за допомогою лямбда-функцій або об'єктів-функцій. Детальніше далі, тут лише приклад:

```
void some_function(){
    auto nested_fn_emul = [](int a, int b){ return a + b; };
    cout << nested_fn_emul(2, 3) << endl;
}
```

[2] GCC підтримує їх як нестандартне розширення: `https://gcc.gnu.org/onlinedocs/gcc/Nested-Functions.html`. Не користуйтеся ним! Навіть Clang, який прагне до максимальної сумісності їх не підтримує.

Аргументи функції

- Під час виклику функції виділяється пам'ять під локальні аргументи функцій і їм присвоюються значення, передані під час виклику.
- Фактично, формальні параметри є локальними змінними функції[3].
- Семантика передачі аргументів та ж, що і при ініціалізації (змінних).
- Аргументи можна передавати **по значенню** (копіюються), **за вказівником чи за посиланням** (як lvalue-посиланням, так і rvalue-посиланням).
- Дані, які функція не повинна змінювати, слід передавати їй по значенню (наприклад, `int x`), у вигляді вказівника на константні дані (`const SomeClass *o`), або константного посилання (`const SomeClass &or`).
  - ◇ Останні два методи дозволяють уникнути копіювання великих об'ємів даних у функцію, при чому — явно забороняючи функції їх модифікувати[4].
- Літерали, константи та аргументи, для яких необхідно виконати перетворення типів у функцію можна передавати лише як `const &` — константні посилання. При цьому, за потреби, буде створено тимчасову змінну.
- Якщо функція не приймає жодних аргументів, вона може про це наголосити явно[5]: `int fn(void);`.

---

[3] Зокрема, їх зміна аж ніяк не впливатиме на дані коду, що викликав функцію. Правда, ці дані можна змінити, якщо передано не-константний вказівник або посилання.

[4] Зрозуміло, що цей захист можна обійти. Але, як то кажуть — захист від дурня безсилий перед винахідливим дурнем.

[5] Щоб не виникало плутанини із старою формою оголошення функцій, без списку формальних параметрів. І в C++ і в найновіших стандартах C це вважається застарілим, але в C поки, ймовірно, варто використовувати.

Значення, що повертаються функцією

- Функція повинна повертати значення того типу, що було вказане у її прототипі – оголошенні.
- Якщо функція не повертає ніякого значення, слід вказати, що вона "повертає" тип **void**.
- Повернення значення відбувається за допомогою оператора **return**, після якого йде значення, яке треба повернути — константа, змінна, вираз, тощо. За потреби виконаються необхідні перетворення типів.
- Функція, яка нічого не повертає, може закінчуватися `return` без аргументів або без return.
- Вихід із функції, що повертає не void, без return, хоча б по одній із гілок – невизначена поведінка. Навіть якщо ця гілка ніколи не виконується!
- В функції може бути декілька `return` — точок виходу.
- Категорично не можна повертати вказівники чи посилання на локальні змінні!

Масиви як аргументи функцій

- Коли в функцію передається масив, насправді в передається лише вказівник на його нульовий елемент.
- ◇ Інформація про розмір масиву втрачається.
- ◇ Зокрема, в тілі функції, sizeof(arr_arg) завжди повертатиме розмір вказівника, ймовірно 4 чи 8.
- ◇ Тому, крім масиву, у функцію слід передавати і його розмір:

```
void our_function (const int arr_arg [], size_t arr_size){
    for(size_t i = 0; i<arr_size; ++i){
        ... якісь дії над arr_arg[ i ]...
    }
}
```

- Якщо передається багатомірний масив — втрачається інформація про розмір першої розмірності[6], але решту розмірностей потрібно передавати:

```
void our_function2 (int arr_arg [][10][15], size_t arr_size );
```

[6] Детальніше див. Бьёрн Страуструп, "Язык программирования C++", специальное издание, додаток B.7.

Приклад передачі масиву

```
void print_array (int arr_arg[], size_t length) {
    for (size_t i=0; i<length; ++i){
        cout << arr_arg[i] << ", ";
    }
    cout << "\n";
}

int main ()
{
    const size_t arr1_size=3;
    int arr1[arr1_size] = {11, 22, 33};
    int arr2[] = {9, 8, 7, 6, 5};
    print_array(arr1, arr1_size);
    print_array(arr2, sizeof(arr2)/sizeof(arr2[0]));
    return 0;
}
```

Відступ: Шаблони та визначення розміру масиву

- Забігаючи наперед, є спосіб визначити розмір масиву за допомогою шаблонів:
- ```
  template <size_t N>
  void print_sample(const int(&arr)[N]){
      std::cout << N << " [";
      for(size_t i = 0; i<N; ++i)
          std::cout << arr[i] << ", ";
      std::cout << " ]" << std::endl;
  }
  ```
- Забігаючи ще сильніше, можна навіть так:

  ```
  template <typename T, size_t N>
  constexpr size_t get_arr_size(const T(&arr)[N]){
      return N;
  }
  template <typename T>
  constexpr size_t get_arr_size(const T& arr){
      return arr.size();
  }
  ```
- Приклад цього коду та аналогічного для двомірних масивів – див. `l04_templ_arr_size_n_fn_args`.
  ```

Статичні локальні змінні

- Локальні змінні функцій створюються кожного разу наново. Між викликами їх вміст втрачається.
- Є можливість створювати спеціальні змінні, які зберігають свої значення між викликами функції – так звані статичні змінні.
- Такі змінні будуть ініціалізуватися лише за першого виклику, а потім зберігатимуть свої значення.

```
int fn1(){
    static int a=0;
    a = a+2;
    return a;
}
```

- Забігаючи наперед: ініціалізація статичних змінних потокобезпечна – компілятор турбується про це.
- ◇ Однак, захист інших звертань до цієї змінної (під час викликів функції з різних потоків) – на вашій совісті. Якщо не потурбуватися (блокуваннями або із використанням TLS) функція, яка не тільки читає такі свої статичні змінні – **не реєнтрантна і не потокобезпечна**.

Статичні функції та глобальні змінні

- Якщо функція призначена лише для виклику в межах одного файлу – є службовою, щоб заборонити лінкуватися із нею з інших файлів, можна оголосити її статичною:

```
static int maximal( int a, int b )
{
    return (a>b) ? a : b;
}
```

- Тоді лінкер не зможе випадково "підхопив", шукаючи іншу функцію з аналогічним прототипом.
- Це ж правило стосується статичних глобальних змінних.
- Даний інструмент походить із C. В C++ є більш просунутий – анонімні простори імен, див. далі.

Рекурсивні функції

- Функції C++ можуть викликати самі себе. Класичний, хоч і *неефективний*, приклад — рекурсивне обчислення факторіалу:

```
unsigned factorial (unsigned n)
{
    if (n>1)
        return n* factorial (n-1);
    else
        return 1;
}
```

- Рекурсія потенційно небезпечна, через загрозу переповнення стеку.
- ◇ Проблема особливо гостра у функціональних мовах. Звідти ж росте потреба у tail call optimization[7].
- ◇ GCC та Clang намагаються цю оптимізацію робити, але часто їм не вдається – і ви не знаєте, працюватиме ваш глибоко рекурсивний код, чи ні. Гірше – він може працювати із оптимізацією і не працювати без.
- ◇ Clang має нестандартне розширення: `[[clang::musttail]]` чи `__attribute__((musttail))`, яке змушує робити цю оптимізацію.
- ◇ Подробиці – плюси, мінуси тощо, див.: `https://blog.reverberate.org/2021/04/21/musttail-efficient-interpreters.html`.

[7] Приклад див.: "Chasing your tail.